

ADR 0021 — Durable operation journal and recovery references

- **Status:** Accepted
- **Date:** 2026-07-25
- **Milestone / issue:** M1.09 — Build a Durable Operation Journal and Recovery References (#62)
- **Supersedes / superseded by:** — (extends ADR 0020's in-memory operation registry across process restarts)

Context

ADR 0020 gave every mutation identity, a lifecycle, and a replayable result — but only for the life of the process. The registry lived in memory: a restart (deploy, crash, OOM kill) erased every record. Two consequences:

1. A client retrying an idempotency key after a restart got a fresh execution, not a replay — the exact hazard ADR 0020 was built to close.
2. "What was HEAD before that operation ran?" had no answer once the process that ran it was gone — no way to offer undo for a mutation whose evidence died with the record.

Decision

Two additions, both best-effort — the journal is a safety net, not a dependency the git operation itself relies on to succeed.

A SQLite journal, one file, process-wide

`persist` writes a row on admission and again on the terminal transition, keyed by the client's own idempotency key. `recover` reads every row back at startup and hands the result to `operations::rehydrate`, so `GET /api/operations/{id}` and idempotency replay both keep working across a restart.

A row left `Running` at recovery time is closed out as `Failed`, not guessed at. The task it named was a `tokio::spawn` ed future belonging to the dead process — a restart doesn't suspend and resume it, it erases it, and there is no way to know from the row alone whether the git command landed. Recovery says so explicitly and leaves the real answer to the staleness gate (ADR 0018) the next time the client acts, the same posture ADR 0019 already takes for a mutation guard's holder dying mid-hold.

Schema carries a `PRAGMA user_version`, versioned from the first migration — a column changing shape is a migration, never a silent `CREATE TABLE IF NOT EXISTS` edit.

Recovery refs under an app-owned namespace

`write_recovery_ref` pins the pre-operation tip a `RecoveryStrategy` names, as a ref under `refs/git-vista/recovery/` — never `refs/heads/` or `refs/tags/`. "Never overwrites a user ref" holds by construction, not by care: no user-chosen branch or tag name can ever resolve into this prefix, because git refs are namespaced by their full path and this path is fixed and app-owned.

A recovery ref outlives the SQLite row that describes it. Restart the server, and the pointer into the object graph is still there even though the row was closed out as interrupted — undo stays offered (gated by the existing `/api/undoables` precondition check) even after a crash.

Redaction

`operations` logs failures with `eprintln!`, and a `GitOperation` can carry free text a user typed — a commit message. Every log line in the durable path goes through `redact_operation`, which keeps only the operation's kind (`commit_on_head`, `push_branch`, ...) and never its fields. The database row itself is deliberately not redacted — persisting operation intent verbatim is the point of the journal — only what reaches the server's own stderr is scrubbed.

Rejected alternative — **redact the database row too**: would make the journal useless for its actual job (reconstructing what an operation was for recovery/support), in exchange for a guarantee SQLite's file permissions already provide (the journal lives beside the session token file, same 0600 posture).

Consequences

- Operation results now survive process restart; idempotency replay is correct across a deploy, not just within one process's lifetime.
- Undo remains answerable after a crash, because the recovery ref's lifetime is decoupled from the SQLite row's.
- The journal is additive and fails open: a journal that can't be opened or written logs and continues — serving repositories never depends on it.
- New failure mode class (journal write/read errors) is entirely non-blocking by design; there is no code path where a durable-layer error surfaces to the client.
- **The durable write now happens *before* the terminal state is published, and that costs latency.** (*Amended 2026-07-28 — #158, PR #160.*) The original implementation called `finish()` first and persisted afterwards, on the reasoning that persistence "adds no latency to what `wait_terminal` is waiting on". That reasoning was wrong in a way that broke this ADR's central promise: `finish()` unblocks every waiter, including the request's own response, so a waiter could observe "done" before the row existed. `recover()` cannot distinguish "not yet journaled" from "orphaned by a crashed process", so its sweep force-failed rows for operations that had genuinely succeeded. The order is now compute (`terminal_status`, read-only) → persist → publish (`finish`). The consequence is deliberate and should not be "optimised" back: **every tracked mutation's response now waits on a SQLite write**, which on a spinning

disk is real per-operation latency. Durability before acknowledgement is the point of this ADR; if that cost ever becomes unacceptable, the answer is a faster durable path, not publishing before persisting.